

The Web: HTTP, HTTPS, DNS

Part I · Foundations — what really happens when you open a web address.

LEARNING OBJECTIVES

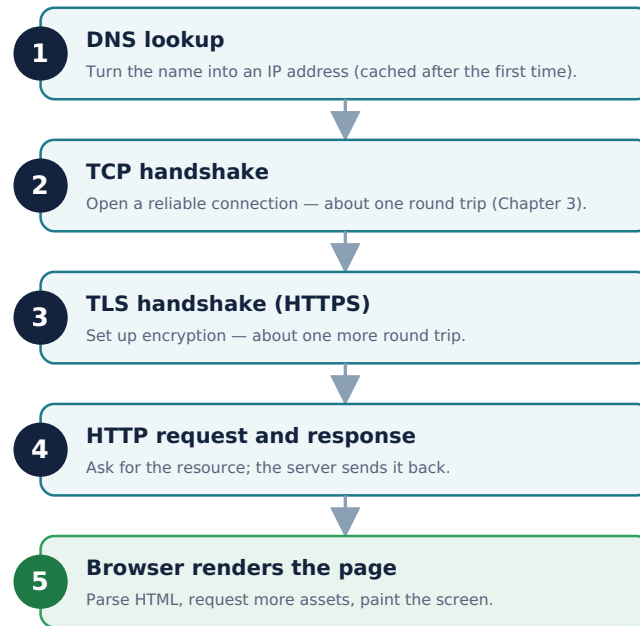
- Trace the full lifecycle of a web request from typing a URL to seeing the page.
- Explain how **DNS** turns a human name into an IP address, and why it is a distributed, cached hierarchy.
- Read and write a raw **HTTP** request and response, and know the key methods and status codes.
- Explain why HTTP is **stateless** and what that means for scaling and for user sessions.
- Explain what **HTTPS/TLS** adds — privacy, integrity, and authentication — and roughly how the handshake works.
- Turn all of this into design rules: caching, connection reuse, TLS termination, and correct method/status usage.

REAL-WORLD ANALOGY — CALLING A BUSINESS

To reach a company you don't start by knowing its phone number — you look it up in a **directory** (that's **DNS**: name → number). Then you dial and connect (the **TCP** call from Chapter 3). Once connected, you both speak a shared, structured language: “Hello, I'd like to order item 12” / “Certainly, here it is” (that's **HTTP**: request and response). And if you're reading out a credit-card number, you switch to a private code only the two of you can understand, so nobody listening on the line can steal it (that's **HTTPS**: encryption). This chapter is that whole phone call, in detail.

4.1 The journey of a web request

When you type `example.com` and press Enter, a precise sequence unfolds in a fraction of a second. Chapter 3 gave us TCP; this chapter fills in the three web-specific pieces — **DNS**, **HTTP**, and **HTTPS** — that surround it. Here is the whole lifecycle:



Steps 1–3 are pure setup. **Later requests reuse the connection and cached DNS, skipping most of it** — which is why the first load feels slow and the rest feel instant.

4.2 DNS — the internet's phone book

Chapter 3 said every machine has an IP address, but you typed a *name*. **DNS (Domain Name System)** is the service that translates a human-friendly name like `example.com` into a machine address like `93.184.216.34`.

WHY THE OBVIOUS SOLUTION FAILED

In the early internet, every computer kept one shared file (`hosts.txt`) listing every name and its address, copied to everyone. That worked for a few hundred machines and became impossible as the internet grew to millions: one file, edited by hand, distributed to the world, is unmaintainable. DNS replaced it with a **distributed, hierarchical, heavily-cached** system that no single party owns and that scales to billions of names.

The hierarchy and how a lookup works

DNS is organized like a tree, resolved by a **recursive resolver** (usually run by your ISP, or a public one like `8.8.8.8`) that does the walking for you:

```

Looking up www.example.com :

browser cache? --> OS cache? --> [ Recursive Resolver ]
                                |
                                "Who handles .com?" -----> [ Root servers ] --> "ask the .com servers"
                                "Who handles example.com?" ----> [ .com TLD servers ] --> "ask example's servers"
                                "What is www.example.com?" ----> [ example.com's      ] --> "93.184.216.34"
                                                                [ authoritative server]
                                resolver caches the answer (for its TTL) and returns the IP.
  
```

Each level answers only its part and points to the next. The **root** servers know where the top-level domains (`.com` , `.org` , country codes) live; the **TLD** servers know the authoritative server for each domain; the **authoritative** server holds the actual records for that domain.

Records, caching, and TTL

DNS stores several record types. The ones to know:

Record	Meaning
A	Name → IPv4 address.
AAAA	Name → IPv6 address.
CNAME	Alias: this name is really another name.
MX	Where to deliver email for this domain.
NS	Which name servers are authoritative for this domain.

Every answer carries a **TTL (Time To Live)** — how long it may be cached. Caching happens at every level (browser, OS, resolver), so most lookups never travel the full tree. This is why DNS is fast in practice, and also why **changing a DNS record isn't instant**: old answers stay cached until their TTL expires ("DNS propagation").

KEY INSIGHT — DNS IS A SYSTEM-DESIGN TOOL, NOT JUST A LOOKUP

Because DNS decides *which IP* a user gets, it's the first place you can steer traffic. A domain can return **different IPs** to spread load across servers (DNS load balancing), or return the address of the **nearest data center** based on the user's location (**GeoDNS**). DNS is the very first entry point of your whole system — which also makes it a critical dependency and a target worth protecting.

4.3 HTTP — the language of the web

Once connected, the browser and server need a shared, structured way to talk. **HTTP (HyperText Transfer Protocol)** is that language. It follows a simple **request-response** model: the client sends a request, the server sends back a response. That's it.

What a request and response actually look like

```

--- REQUEST ---
GET /articles/42 HTTP/1.1      <- method, path, version
Host: example.com             \
User-Agent: Mozilla/5.0        | headers (key: value)
Accept: text/html              /
                               <- blank line ends the headers

(no body for a GET)

--- RESPONSE ---
HTTP/1.1 200 OK                <- version, status code, reason
Content-Type: text/html        \
Content-Length: 1256           | headers
Cache-Control: max-age=3600    /

```

```

    <- blank line
<html> ... the page ... </html>  <- body

```

Methods: what you want to do

Method	Purpose	Safe?	Idempotent?
GET	Read a resource (should never change data)	Yes	Yes
POST	Create something / submit data	No	No
PUT	Replace a resource entirely	No	Yes
PATCH	Update part of a resource	No	No (usually)
DELETE	Remove a resource	No	Yes

Safe means it only reads (no side effects). **Idempotent** means doing it multiple times has the same effect as doing it once — directly connected to Chapter 3’s lesson that you can safely retry idempotent calls after a network timeout, but not unsafe ones like a bare `POST`.

Status codes: how it went

Range	Meaning	Common examples
2xx	Success	200 OK, 201 Created
3xx	Redirect (go elsewhere)	301 Moved Permanently, 302 Found
4xx	Client error (your request was wrong)	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests
5xx	Server error (the server failed)	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

HTTP is stateless — and that’s on purpose

KEY INSIGHT — STATELESSNESS ENABLES HORIZONTAL SCALING

By default HTTP is **stateless**: the server does not remember anything about you between requests — each request stands alone and carries everything needed to handle it. This feels limiting, but it’s a superpower for scaling: since no request depends on server memory of a previous one, *any* server can handle *any* request. That’s what lets you put many identical servers behind a load balancer (Chapters 7 and 10). The price is that **identity must travel with each request** — carried in a cookie or token — which is exactly what sessions and JWTs (Chapter 41) provide.

HTTP versions, each cutting latency

Version	Key improvement
HTTP/1.1	Persistent connections (keep-alive) — reuse one TCP connection for many requests instead of reconnecting each time.
HTTP/2	Multiplexing — many requests share one connection at once, plus header compression. Fewer round trips.

HTTP/3 Runs over QUIC (UDP, Chapter 3) to avoid TCP's head-of-line blocking and set up faster.

Notice the theme: every version exists to **reduce round trips and latency** — the same obsession we met in Chapters 2 and 3.

4.4 HTTPS & TLS — the web, secured

Plain HTTP travels in **cleartext**: anyone on the path — the coffee-shop Wi-Fi, your ISP, any router in between — can read it and even alter it. That's catastrophic for passwords, payments, or private messages. **HTTPS** fixes this by running HTTP on top of **TLS (Transport Layer Security)**.

HTTPS gives three guarantees:

- **Encryption (privacy)**: eavesdroppers see only scrambled gibberish.
- **Integrity**: if anyone tampers with the data in transit, it's detected.
- **Authentication**: you can be sure you're really talking to `example.com` and not an imposter — proven by a **certificate**.

How TLS works (the short version)

After the TCP handshake, a **TLS handshake** runs. In simple terms:

1. The server presents a **certificate** proving its identity, signed by a **Certificate Authority (CA)** that your browser already trusts.
2. Using public-key cryptography (a public key anyone can use to encrypt, and a private key only the server holds), the two sides **securely agree on a shared secret key** without ever sending it in the clear.
3. From then on they use that shared key for fast **symmetric encryption** of all the actual data.

(The cryptography is covered fully in Chapter 43; here the concept is enough.) The **chain of trust** is what makes the padlock meaningful: your browser trusts a small set of CAs, those CAs vouch for the certificate, so you can trust the site. Free providers like Let's Encrypt made HTTPS universal.

THE COST — AND HOW SYSTEMS PAY IT

The TLS handshake adds another **round trip of latency** on top of TCP (step 3 in the lifecycle diagram). That's more reason to **reuse connections** (Chapter 3). Modern **TLS 1.3** trims the handshake to about one round trip, with even faster resumption for repeat visits. In large systems, TLS is usually **terminated at the edge** — a load balancer or API gateway (Chapters 7 and 9) decrypts incoming HTTPS so backend services can work in plain HTTP inside the trusted network, saving them the crypto cost.

4.5 The full lifecycle, one more time

You can now read the whole story end to end. Type `https://example.com` and press Enter: **(1)** DNS resolves the name to an IP (often from cache); **(2)** a TCP connection opens to that IP on port 443; **(3)** a TLS handshake secures the channel and verifies the certificate; **(4)** the browser sends an HTTP `GET` request and the server responds with the page and a status

code; **(5)** the browser renders it, firing more requests for images, CSS, and scripts — most of which reuse the now-open, already-secured connection. Everything you learned in Part I meets in that single sequence.

Common mistakes

WATCH OUT

- Forgetting HTTP is **stateless** — assuming the server “remembers” a user without a cookie or token.
- Using **GET** for actions that change data. GET should be safe; it also gets cached and pre-fetched, causing surprise side effects.
- Retrying non-idempotent methods (a bare POST) after a timeout — risking duplicate actions.
- Expecting DNS changes to take effect instantly, ignoring **TTL** and caching.
- Opening a new connection (and TLS handshake) per request instead of reusing one.
- Sending sensitive data over plain HTTP instead of HTTPS.
- Misusing status codes — e.g. returning `200 OK` with an error hidden in the body.
- Not setting **Cache-Control** on cacheable responses, wasting round trips.

Interview questions

1. Walk me through exactly what happens, step by step, when you type a URL and press Enter.
2. How does DNS resolution work? Explain the roles of root, TLD, and authoritative servers, and of caching/TTL.
3. Why is HTTP stateless, and how do we keep users logged in on top of it?
4. Compare GET, POST, PUT, PATCH, DELETE. Which are safe, and which are idempotent? Why does that matter for retries?
5. What three guarantees does HTTPS provide, and roughly how does the TLS handshake achieve them?
6. What is a Certificate Authority and the “chain of trust”? Why does your browser trust a site?
7. Why might a DNS record change take hours to be seen by all users?
8. What problems do HTTP/2 and HTTP/3 solve compared with HTTP/1.1?

Hands-on exercises

1. Run `dig` or `nslookup` on a domain. Identify the A record and its TTL; run it again and note that the second answer comes from cache.
2. Use `curl -v https://example.com` and find, in the output, the TLS handshake, the certificate issuer, the request headers, and the response status line.
3. Trigger and record four different status codes from real sites: a 200, a 404, a 301/302 redirect, and (if you can) a 403 or 401.

- Click the padlock in your browser on any HTTPS site and read the certificate: who issued it and when it expires.
- Classify each as safe, idempotent, both, or neither: GET, POST, PUT, DELETE.

Mini project

SPEAK HTTP BY HAND

Using `curl`, make three raw requests and read the full responses (headers included, via `-i` or `-v`): a normal `GET` to a real page; a `GET` to a path that doesn't exist (observe the 404); and a request with a custom header you add yourself. Then make one HTTPS request with `-v` and locate the TLS handshake and certificate details in the output. You'll see, concretely, the request/response structure and the security layer this chapter described — and how much of the time is setup.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE WEB LAYER

In your design notebook, sketch the core **HTTP API surface**: a handful of endpoints with their methods and expected status codes (e.g. `POST /posts` → 201, `GET /users/{id}` → 200 or 404, `DELETE /posts/{id}` → 204). Mark which endpoints are **idempotent**. Decide your **caching** policy (which responses are cacheable, and their `Cache-Control`). Note your **DNS** strategy (single region now; GeoDNS later for multi-region, foreshadowing Part VIII). Confirm everything is **HTTPS**, with TLS terminated at the edge. This API surface becomes the front door for every later part of your design.

Chapter summary

- A web request's lifecycle is: **DNS lookup** → **TCP handshake** → **TLS handshake** → **HTTP request/response** → **render**. The first three are setup that later requests mostly skip.
- DNS** is a distributed, cached hierarchy (root → TLD → authoritative) that maps names to IPs; **TTL** controls caching and explains propagation delay. It's also a traffic-steering tool (DNS/GeoDNS).
- HTTP** is a stateless request-response language with **methods** (GET/POST/PUT/PATCH/DELETE), **status codes** (2xx-5xx), and headers. Statelessness is what enables horizontal scaling; identity must travel in each request.
- Method **safety and idempotency** determine what's safe to retry after a network failure.
- HTTPS/TLS** adds encryption, integrity, and authentication via certificates and a chain of trust — at the cost of a handshake round trip, usually terminated at the edge.
- Every HTTP version and TLS improvement chases the same goal from Part I: **fewer round trips, lower latency**.

COMING NEXT

Chapter 5 — Latency, Throughput & the Numbers Every Engineer Knows. We'll consolidate Part I by turning all these round trips and hardware speeds into hard numbers —

the back-of-the-envelope figures you'll use to estimate whether a design needs one server or a thousand, closing out the Foundations and opening the door to scaling in Part II.